Project Phase 2

CMPT 276: Introduction to Software Engineering
Simon Fraser University

Group 1

Mohamad Osman	301590514
Brian Chan	301439177
Keith	301625387
Daniil Bugakov	301540700
Dev Tomar	301548576

March 11, 2026

Contents

1 Overall Approach to Implementing the Game **1**

2 Adjustments and Modifications from the Phase 1 Design **1**

3 Management Process and Division of Roles **3**

4 External Libraries Used **3**

5 Biggest Challenges Faced During This Phase **4**

1 Overall Approach to Implementing the Game

The goal of Phase 2 was to convert the design created in Phase 1 into a working implementation of the game. The team approached this phase by first building the core board and movement systems, then gradually adding gameplay features such as traps, collectibles, enemy behavior, timed rewards, and graphical interface elements.

The game was implemented as a grid-based system where the board layout is loaded from a predefined map. The Board class acts as the main game engine and is responsible for loading the map, storing objects placed on the board, updating the game state, and processing interactions between the player, enemies, and items. Because of this, the Board class serves as the central component where most game systems interact.

The board itself is represented using Cell objects, where each cell corresponds to a tile on the game map. When the game starts, the board layout is interpreted and the correct objects are created at each location. The BoardFactory class handles the creation of the board and ensures that the correct objects are placed based on the map configuration.

Character movement was implemented using shared support classes. The Position class stores coordinates on the grid, while the MovingEntity class provides a structure for objects that move across the board. This structure is used by both the Thief and the Guard classes. The thief represents the player-controlled character, while the guard acts as the enemy that attempts to catch the player.

The Guard enemy uses a Breadth-First Search (BFS) pathfinding algorithm to dynamically compute the shortest path toward the thief while navigating around walls and other obstacles on the board.

Rather than using complex switch statements, rewards and traps extend an Interactable abstract class. Rewards such as CashStack increase the player's balance when collected, while traps such as InkDart and GlueTrap apply their own negative effects such as reducing the balance or freezing movement.

The RareDiamond represents a high-value reward, while a separate timer manages when the diamond randomly appears and disappears without affecting the main gameplay systems.

A graphical interface was implemented using a separate Interface class. This class manages all non-gameplay screens such as the main menu, pause screen, win screen, game over screen, and tutorial. It also controls the in-game HUD which displays score and playtime information. The Board class maintains responsibility for gameplay logic and rendering of the play area.

Overall, the implementation process followed an incremental approach. The team first ensured that the board and movement systems worked correctly, then added additional gameplay features and interface functionality. This approach allowed the game to be tested continuously as new systems were added.

2 Adjustments and Modifications from the Phase 1 Design

The Phase 1 UML diagram provided the initial design structure for the project. However, several adjustments were made during implementation to simplify the code and improve

modularity. While the overall architecture remained similar to the original design, the transition from UML diagrams to working code required practical refinements.

The initial design proposed several classes for different parts of the core game system. The GameEngine controlled the game loop and game logic, InputHandler processed player inputs, ElapsedTimer tracked playtime and Exit validated the win condition. In the final implementation, however, these classes were omitted and their responsibilities were consolidated into the Board class to reduce architectural complexity.

A BoardFactory was introduced to generate playable levels following the factory pattern and enabling easier support for multiple levels in future expansions of the game.

CellType enumeration was originally planned to classify board tiles such as walls, exits, and traps. In the final implementation this enumeration was removed and cells are instead managed through dedicated collections representing different tile types.

The Cell class was also simplified compared to the initial design. Originally it was intended to store cell type, rewards, traps, and entity occupancy. In the final implementation, the Cell class primarily represents a tile used for positioning and rendering on the board.

During the Phase 1 design the board was planned as a rigid array where each cell maintained a state enumeration, and the engine would evaluate those states to trigger gameplay events. However, this approach resulted in overly complex switch statements during early development. To address this issue, the team implemented the Strategy pattern using the Interactable abstract class.

For example, the CashStack class represents a collectible reward that increases the player's balance, while traps such as InkDart and GlueTrap encapsulate their own negative effects. By allowing each item class to define its own interaction behavior, the Board class only needs to trigger the corresponding method when a collision occurs. This significantly simplified the game engine.

Another adjustment was the introduction of the ItemFactory class. While the UML design included multiple item types, it did not define a centralized mechanism for creating them. The ItemFactory now manages object creation when the board loads the level configuration.

The implementation also introduced a DiamondSpawner system. The UML design included collectible rewards, but the concept of temporary dynamic rewards was expanded during development. The DiamondSpawner uses Java Swing timers to periodically spawn rare diamonds and remove them after a short period of time.

Another improvement was the addition of the PlaytimeManager class which manages time-related gameplay features such as playtime tracking and timer-based events.

The movement system was also refined. The UML design suggested shared movement behaviour between characters. This was implemented using the MovingEntity base class and the Position class, which together provide reusable movement logic.

The Guard class required additional development compared to the UML design. Instead of random movement, the guard uses a BFS pathfinding algorithm to chase the thief while navigating around walls.

Finally, the Interface class was introduced to separate menu and UI functionality from gameplay systems. This improves code organization and keeps gameplay logic separate from presentation logic.

Overall, the transition from UML diagrams to working code required refining the architecture while preserving the overall structure of the original design.

3 Management Process and Division of Roles

The management process for this phase involved dividing the project into several major subsystems and assigning team members responsibility for specific components. This approach allowed different parts of the game to be developed simultaneously while maintaining a consistent overall design across the project.

The team followed a milestone-based development process during Phase 2. The first milestone focused on implementing the core board structure and basic player movement. Once these systems were functioning correctly, the team expanded the project by adding additional gameplay features such as items, traps, enemy behaviour, and graphical interface elements.

Mohamad worked primarily on the Thief class and player movement system. This included implementing movement controls and ensuring that the player cannot move through walls or outside the boundaries of the board.

Brian focused on static rewards and trap interactions, implementing collectible and penalty behaviour through the Strategy pattern using the Interactable abstract class.

Keith worked on the graphical interface and animation systems, ensuring that the user interface and user experience were smooth and responsive. Keith also designed visual assets for the game using Figma.

Daniil worked on core board systems, including components related to board generation, board structure, and initialization of the game environment.

Dev focused on enemy behaviour and shared movement logic, including the implementation of the Guard class and movement support through classes such as MovingEntity and Position.

As these systems were developed, the team regularly tested how the different components interacted with each other. Integration testing ensured that player movement, item interactions, enemy behaviour, and interface updates all functioned correctly together within the full game system.

4 External Libraries Used

The project was implemented using standard Java libraries.

Java Swing

Swing was used to build the main application window, manage UI components, and handle timer-based game updates. It provided the necessary functionality for rendering the game board and managing interface screens.

Java AWT

AWT was used for lower-level graphical tasks including drawing elements, rendering colours, managing fonts, and handling keyboard and mouse input events.

These libraries were chosen because they are included in the standard Java environment and provide sufficient functionality for implementing a grid-based graphical game without requiring additional dependencies.

5 Biggest Challenges Faced During This Phase

One of the biggest challenges during this phase was managing the interactions between multiple gameplay systems. The board contains the player, enemies, collectible items, traps, and special rewards. Ensuring that all of these systems worked together consistently required careful coordination between different classes.

Another challenge was determining how responsibilities should be divided between classes. Early in development, too much logic accumulated inside the Board class. The project required refactoring so that item behaviour, reward spawning, and time-based systems were handled by specialized classes.

Implementing the guard enemy behaviour was another major challenge. The guard needed to move intelligently toward the player while navigating around walls and obstacles. This required implementing BFS pathfinding and integrating it with the board structure.

Testing and integration also presented difficulties. Systems that worked correctly individually sometimes required adjustments once integrated with the rest of the game.

Another challenge was translating the Phase 1 UML design into practical implementation decisions. While the UML diagrams provided a conceptual structure, the coding process revealed situations where additional helper classes and architectural changes were required.

The team also encountered version control challenges early in development. Some features were initially developed in isolated branches without proper coordination, which caused merge conflicts and incompatible code. To resolve this issue the team introduced a centralized development branch and began merging updates more frequently.

Despite these challenges, the team successfully implemented the core systems required for a fully playable version of the game.